

GiftKeeper

Final Academic Project Report for the Automated Software Testing Course

Student

Alhussein Ghader

Course

Automated Software Testing

Academic Year

2025–2026

1. Introduction

GiftKeeper is a modular Java application developed to manage people, occasions, gift ideas, budgets, and gift tracking.

The project was designed with a strong focus on modern software engineering practices and automated testing methodologies studied during the Automated Software Testing course.

The implementation emphasizes:

- Test-Driven Development (TDD)
- Multi-layer automated testing
- Build Automation
- Continuous Integration
- Code Quality Analysis
- Docker-based reproducible environments
- Maintainable and testable software architecture

The goal of the project is not only to build a functional application, but also to demonstrate software reliability, maintainability, and continuous quality verification through automated tools and testing strategies.

2. Project Objectives

The project was developed to achieve the following educational and technical objectives:

- Apply Test-Driven Development principles
 - Use JUnit 5 for automated testing
 - Use AssertJ for expressive assertions
 - Use Mockito for dependency isolation
 - Use Maven for Build Automation
 - Organize the project using a Maven Multi-Module architecture
 - Use Git and GitHub for version control
 - Use GitHub Actions for Continuous Integration
 - Use JaCoCo for code coverage
 - Use PIT Mutation Testing
 - Use SonarCloud for code quality analysis
 - Use Docker/Testcontainers for database testing
 - Implement real Integration Tests using PostgreSQL
 - Implement End-to-End tests
 - Provide both GUI and CLI interfaces
 - Maintain a clean and modular architecture
-

3. System Overview

GiftKeeper is designed to manage the organization of gifts and occasions in a simple but extensible way.

The application allows users to:

- Manage people
- Create occasions
- Add gift ideas
- Define prices and descriptions
- Track gift statuses
- Maintain gift history
- Apply reminder policies
- Use the application through CLI or GUI
- Persist data using PostgreSQL

The project domain was intentionally kept understandable so that the main focus remains on testing, architecture, automation, and software quality.

4. Technologies Used

Technology	Purpose
Java 17	Main programming language
Maven	Build Automation and Multi-Module Management
JUnit 5	Unit and Integration Testing
AssertJ	Fluent Assertions
Mockito	Mocking and dependency isolation
Hibernate ORM	ORM and persistence management
JPA	Persistence abstraction
PostgreSQL	Relational database
Testcontainers	Dockerized testing
Docker	Reproducible environments
JaCoCo	Code coverage
PIT	Mutation Testing
GitHub Actions	Continuous Integration
SonarCloud	Code quality analysis
Swing	Desktop graphical interface
BDD/Cucumber	Behavioral testing

5. Project Architecture

The project follows a layered modular architecture designed to maximize:

- Maintainability
- Testability
- Separation of concerns
- Extensibility
- Reduced coupling between layers

The system is divided into several independent Maven modules:

```
giftkeeper-parent
├── giftkeeper-domain
├── giftkeeper-app
├── giftkeeper-persistence-api
├── giftkeeper-persistence-jpa
├── giftkeeper-cli
├── giftkeeper-gui
└── giftkeeper-bdd
```

6. Module Description

6.1 giftkeeper-domain

Contains the core business logic of the application.

This module includes:

- Person
- Occasion
- GiftIdea
- GiftStatus
- Budget
- ReminderPolicy
- GiftHistoryEntry

This layer is completely independent from:

- databases
- frameworks
- user interfaces
- Hibernate

This separation allows fast and isolated testing.

6.2 giftkeeper-app

Contains application services and use cases.

Responsibilities include:

- workflow orchestration
 - connecting repositories with domain logic
 - coordinating business operations
-

6.3 giftkeeper-persistence-api

Contains repository interfaces only.

Examples:

- PersonRepository
- OccasionRepository
- GiftIdeaRepository

This layer applies Dependency Inversion and reduces coupling between application logic and persistence implementation.

6.4 giftkeeper-persistence-jpa

Implements persistence using:

- Hibernate
- JPA
- PostgreSQL

This layer is responsible for real data persistence.

6.5 giftkeeper-cli

Provides a simple command-line interface for interacting with the application.

6.6 giftkeeper-gui

Provides a Swing-based desktop graphical interface.

The GUI layer is intentionally separated from business logic to improve maintainability and testability.

6.7 giftkeeper-bdd

Contains high-level Behavior-Driven tests.

6.8 giftkeeper-e2e

Contains real End-to-End tests using PostgreSQL running inside Docker containers.

7. Architectural Design Decisions

7.1 Why Multi-Module Maven?

The project uses a Multi-Module Maven structure to achieve:

- separation of responsibilities
 - maintainability
 - modularity
 - easier testing
 - independent layers
-

7.2 Why Separate Domain from Persistence?

Business logic should not depend on databases or ORM frameworks.

This separation allows:

- faster testing
 - easier maintenance
 - lower coupling
 - reusable business logic
-

7.3 Why Repository Interfaces?

Repository interfaces are used to apply Dependency Inversion.

The Application Layer depends on abstractions instead of PostgreSQL directly.

This allows using:

- InMemory repositories
- Mock repositories
- JPA repositories

depending on the type of testing required.

7.4 Why Use InMemory Repositories?

InMemory repositories are intentionally used only in Unit Tests.

Their purpose is to:

- support fast TDD cycles
- speed up Unit Tests
- isolate components
- avoid unnecessary database dependency during isolated testing

Integration and End-to-End tests use real PostgreSQL containers through Docker/Testcontainers.

7.5 Why Use Testcontainers?

Testcontainers provide realistic Docker-based testing environments.

Instead of using artificial or manually configured databases, PostgreSQL containers are started automatically during tests.

This guarantees:

- realistic testing
 - reproducible environments
 - isolated execution
 - fewer configuration problems
-

8. Testing Strategy

The project follows a multi-layer testing strategy.

The goal is not only to maximize coverage percentages, but also to verify:

- behavior correctness
 - architectural quality
 - robustness
 - system reliability
-

9. Unit Testing

Unit Tests verify isolated components.

Examples include:

- PersonTest
- GiftIdeaTest
- BudgetTest
- ReminderPolicyTest

The tests verify:

- input validation
- edge cases
- exceptions
- state transitions
- business rules

The project uses:

- JUnit 5
- AssertJ

Mockito is used when dependency isolation is required.

10. Integration Testing

Integration Tests verify collaboration between:

- application services
- Hibernate
- JPA
- PostgreSQL

These tests use real PostgreSQL databases through Docker/Testcontainers.

The tests validate:

- JPA mappings
 - persistence behavior
 - data retrieval
 - real workflows
-

11. End-to-End Testing

End-to-End tests simulate complete application workflows.

The tests verify:

- entity creation
- persistence
- retrieval
- full workflow consistency

This demonstrates that the system works as a fully integrated application.

12. GUI Testing

The project includes a Swing-based GUI.

The interface was designed so that business logic remains outside graphical components, improving maintainability and testability.

13. BDD Testing

BDD tests describe system behavior from a higher-level user-oriented perspective.

This improves:

- readability
- behavioral documentation

- scenario verification
-

14. Code Coverage

JaCoCo is used to measure:

- Line Coverage
- Branch Coverage

The project achieves high coverage while keeping tests maintainable and understandable.

15. Mutation Testing

PIT Mutation Testing is used to evaluate test strength.

PIT intentionally modifies source code and verifies whether tests can detect the behavioral change.

This provides stronger confidence than traditional coverage metrics alone.

Mutation testing focuses mainly on:

- domain logic
- application services

because these layers contain the most important business behavior.

Persistence adapters are validated through Integration and End-to-End tests instead.

16. Docker and Testcontainers

Docker is used indirectly through Testcontainers.

During Integration and End-to-End tests, PostgreSQL containers are automatically started.

This guarantees:

- reproducible environments
 - isolated testing
 - CI compatibility
 - reduced configuration complexity
-

17. Build Automation with Maven

Maven is used to automate the entire build and testing lifecycle.

The main command is:

```
mvn clean verify
```

This command performs:

- project cleanup
 - compilation
 - automated testing
 - integration testing
 - final project verification
-

18. Continuous Integration

GitHub Actions is used for Continuous Integration.

The pipeline automatically executes:

- compilation
- automated tests
- code quality analysis
- mutation testing

on every push.

This guarantees continuous verification of project stability and quality.

19. Code Quality Analysis

SonarCloud is integrated for static code analysis.

It verifies:

- code quality
- code smells
- duplication
- maintainability
- coverage

The project was continuously cleaned and improved until all relevant issues were resolved.

20. Git and GitHub Usage

Git is used for version control.

The project is hosted on GitHub to support:

- version tracking
 - CI integration
 - SonarCloud integration
 - project evolution tracking
-

21. Running the Project

Full Build

```
mvn clean verify
```

Run Mutation Testing

```
mvn -pl giftkeeper-domain,giftkeeper-app pitest:mutationCoverage
```

Run CLI

```
mvn -pl giftkeeper-cli exec:java
```

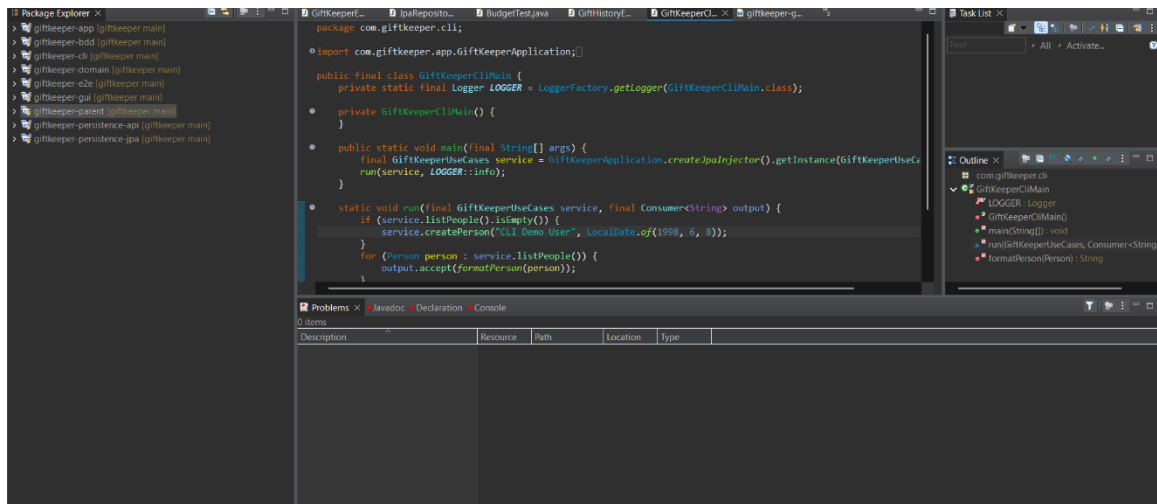
Run GUI

```
mvn -pl giftkeeper-gui exec:java
```

22. Screenshots

The final report should include screenshots of:

1. Eclipse project import
2. Eclipse Problems View without errors



3. Maven BUILD SUCCESS

```
[INFO] -----
[INFO] Reactor Summary for GiftKeeper Parent 1.0.0-SNAPSHOT:
[INFO]
[INFO] GiftKeeper Parent ..... SUCCESS [ 0.307 s]
[INFO] giftkeeper-domain ..... SUCCESS [ 9.705 s]
[INFO] giftkeeper-persistence-api ..... SUCCESS [ 0.327 s]
[INFO] giftkeeper-persistence-jpa ..... SUCCESS [ 27.747 s]
[INFO] giftkeeper-app ..... SUCCESS [ 10.800 s]
[INFO] giftkeeper-gui ..... SUCCESS [ 12.516 s]
[INFO] giftkeeper-cli ..... SUCCESS [ 7.911 s]
[INFO] giftkeeper-bdd ..... SUCCESS [ 5.843 s]
[INFO] giftkeeper-e2e ..... SUCCESS [ 22.795 s]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:38 min
[INFO] Finished at: 2026-05-28T19:32:45+03:00
[INFO] -----
```

4. Successful GitHub Actions workflow

hussienghader / giftkeeper

CodeIssuesPull requestsActionsProjectsWikiSecurity and qualityInsightsSettings

Actions

New workflow

giftkeeper CI

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

All workflows

Showing runs from all workflows

Filter workflow runs

3 workflow runs

Workflow

Event

Status

Branch

Actor

Clean Eclipse warnings

GiftKeeper CI #19: Commit 6ac7307 pushed by hussienghader

main

36 minutes ago

1m 50s

...

Final reviewed academic submission cleanup

GiftKeeper CI #18: Commit e61e3be pushed by hussienghader

main

May 27, 5:59 PM GMT+3

1m 42s

...

Final Sonar cleanup

GiftKeeper CI #14: Commit e83de22 pushed by hussienghader

main

May 27, 3:29 PM GMT+3

1m 35s

...

5. SonarCloud Quality Gate

hussienghader / GiftKeeper / Overview

Overview

Public · No tags · 1.6k Lines of Code · Last analysis 35 minutes ago

View

Project health dashboard

See your project's branch health at a glance by exploring trends and risk breakdowns.

Quality Gate Status

Overall code · Status: Open

✓

Passed

All conditions passed

Open Issues

Overall code · Status: Open

0

Duplications

Overall code

2.7%

⬆️ +8.0% vs last 30 days

Coverage

Overall code

100%

⬆️ No change vs last 30 days

Security snapshot

Security Rating

Overall code

A

Security Issues

Overall code · Software quality: Secu...

0

⬆️ No change vs last 30 days

Open Security Issues by Severity

Overall code · Software quality: Security · Slice by: Severity

No data available to display

6. PIT Mutation Testing report

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
7	100% 131/131	100% 77/77	100% 77/77

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.giftkeeper.domain	7	100% 131/131	100% 77/77	100% 77/77

Report generated by [PIT](#) 1.16.1

Enhanced functionality available at [arcmutate.com](#)

Pit Test Coverage Report

Package Summary

com.giftkeeper.domain

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
7	100% 131/131	100% 77/77	100% 77/77

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
Budget.java	100% 14/14	100% 12/12	100% 12/12
GiftIdea.java	100% 44/44	100% 25/25	100% 25/25
GiftStatus.java	100% 9/9	100% 5/5	100% 5/5
Occasion.java	100% 21/21	100% 11/11	100% 11/11
Person.java	100% 24/24	100% 11/11	100% 11/11
ReminderEngine.java	100% 7/7	100% 3/3	100% 3/3
ReminderPolicy.java	100% 12/12	100% 10/10	100% 10/10

Report generated by [PIT](#) 1.16.1

7. Running GUI

The screenshot shows the GiftKeeper application interface. At the top, there's a form for adding a person with fields for 'Person name', 'Birth date (YYYY-MM-DD)' (set to 1998-06-08), 'Person id', 'BIRTHDAY' (set to 2026-06-08), and 'Birthday'. There are buttons for 'Add person' and 'Add occasion'. Below this is a section for 'Add gift' with a 'personId' dropdown and an 'occasionId' dropdown. A table below shows a list of gifts with columns: Id, Name, Birth date, Id, Person, Type, Date, Description, Id, Person, Occasion, Title, Price, and Status. The table contains four rows of data.

Id	Name	Birth date	Id	Person	Type	Date	Description	Id	Person	Occasion	Title	Price	Status
36caf680-d0b5-4aec-ad	alhasan Ghader	1990-08-18	4b0a8d9d-df	1020de1d-74	BIRTHDAY	2026-06-08	Graduation ...	a8028c0a-	db48b6c4-	0e5c1b47-	Graduatio...	1150.00	BOUGHT
8bce285-fe6e-4678-96	Alhussein	1998-06-08	0e5c1b47-f6	db48b6c4-bd	GRADUATI...	2026-06-08	Graduation ...	0af3db52-	1020de1d-	4b0a8d9d-	iphone	1000.00	GIFTED
1020de1d-74dc-4071-b	AlhusseinGhader	1998-06-08											
1c5def77-65ea-4552-87	Luciana Bordiglia	1961-08-18											
8f4c252b-4ba0-48c2-8a	Luciana Bordiglia	1961-08-18											
db48b6c4-bd78-4d34-9	Zein Ghader	1998-06-08											

8. Running CLI

```
May 28, 2026 8:53:22 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001001: Connection properties: {password=****, user=giftkeeper}
May 28, 2026 8:53:22 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001003: Autocommit mode: false
May 28, 2026 8:53:22 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PooledConnect
INFO: HHH10001115: Connection pool size: 20 (min=1)
May 28, 2026 8:53:25 PM org.hibernate.engine.transaction.jta.platform.internal.JtaPlatformInitiator initiateService
INFO: HHH0000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
May 28, 2026 8:53:25 PM org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIso
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironment
[main] INFO com.giftkeeper.cli.GiftKeeperCliMain - 66caf680-d0b5-4aec-ad99-359a9f6c36e7 | alhasan Ghader | 1990-08-18
[main] INFO com.giftkeeper.cli.GiftKeeperCliMain - 88bce285-fe6e-4678-96b9-ef2735bef901 | Alhussein | 1998-06-08
[main] INFO com.giftkeeper.cli.GiftKeeperCliMain - 1020de1d-74dc-4071-b698-ef7bb8beaa4b | AlhusseinGhader | 1998-06-08
[main] INFO com.giftkeeper.cli.GiftKeeperCliMain - dc5def77-65ea-4552-87c6-1eb58e6e2fa6 | Luciana Bordiglia | 1961-08-18
[main] INFO com.giftkeeper.cli.GiftKeeperCliMain - 8f4c252b-4ba0-48c2-8a4e-c8e57d933fa5 | Luciana Bordiglia | 1961-08-18
[main] INFO com.giftkeeper.cli.GiftKeeperCliMain - db48b6c4-bd78-4d34-929d-573f079e45ce | Zein Ghader | 1998-06-08
```

23. Challenges Encountered

Several technical challenges were encountered during development, including:

- separating persistence from business logic
- configuring Hibernate and PostgreSQL correctly
- integrating Docker/Testcontainers into automated tests
- achieving strong mutation testing results
- maintaining modularity across Maven modules

These challenges significantly improved the quality of the final architecture.

24. Conclusion

GiftKeeper demonstrates a practical and advanced application of the concepts studied in the Automated Software Testing course.

The project combines:

- modular architecture
- automated testing
- mutation testing
- Dockerized integration testing
- Continuous Integration
- static code analysis

within a maintainable and extensible software design.